Affordable-Housing Analytics: Bengaluru House-Price Prediction

Komal

Department of Computer Science, Ramanujan College, University of Delhi, New Delhi, India | Roll No.: 24020570037 | E-mail: komal.1@ramanujan.du.ac.in

Course: Machine Learning (B.Sc. (H) Computer Science, Semester V) | Supervisor: Dr. Nikhil Kumar Rajput | Problem No. 47 from Project_Allocation.xlsx | Article type: Original Research Article

Abstract. Housing affordability is an issue in cities because home prices change a lot based on where the house is howbigit is, how many rooms it has and other features. This study looks at house prices in Bengaluru. Tries to find homes that cost less than a set limit—₹45 lakh. To do this we built a machine-learning systemthat goes through a process: cleaning the datamaking new features checking what the data shows filtering out extreme values turning categories into numbers scalingthenumbersand testing different models.We tested eight regression models to predict house prices: Linear Regression, Ridge, Lasso, Decision Tree, k-Nearest Neighbours, Support Vector Regression, RandomForest and Multilayer Perceptron. We also tested seven classification models to see which homes fall below the ₹45 lakh affordability mark. For the regression models we used Mean Absolute Error, Root Mean Squared Error and the coefficient of determination to check how well they did. For classificationwe looked at accuracy, precision, recall, F1-score and ROC-AUC.We also used Principal Component Analysis to see if reducing the number of features helps. We checked how strong the models are across areas and different BHK types (like 2BHK, 3BHK). We included a baseline: the median price per square foot, fromthe training data to compare against. The results show howwell the models work and also point outthe limits of using listing datato study affordability.

Keywords: Housing affordability, Bengaluru, house-price prediction, machine learning, regression, classification, principal component analysis, urban analytics.

1. Introduction

Housing is a part of what families spend money on and how easy it is to buy a home is a big issue in cities that are growing fast. Bengaluru is a place in India where a lot of people work in technology and find jobs. In Bengaluru the prices of homes can be very different from one area to another. From one type of house to another. This difference means that being able to estimate home prices accurately is helpful when looking at the housing market in the beginning.Traditional ways of valuing homes usually depend on homes what experts think and the cost per square foot. These methods can give some ideas but they might not completely show how different features of a home affect the price. Machine learning gives another way by learning from home data and seeing if it can guess the price of a home that hasn't been seen before.This study looks at using machine learning with home data from Bengaluru. It looks at two related things: predicting the price of a home that is listed and deciding if a home is under a certain price limit. The second part is meant to help check if a home's a good price, not to say if it is affordable for a specific family.The main goals of the study are:

1. To look at missing data how prices are spread out. How different features of a home are connected.

2. To compare machine learning methods for predicting prices using the same way to check how well they do.

3. To split homes into those that're under a set price of ₹45 lakh and those that are not.

4. To see how using Principal Component Analysis affects how well the models work.

5. To look at how the mistakes in predictions change from one place to another and, for types of homes.

The study gives a way to predict home prices and check if they are affordable that can be repeated. The results only apply to the data used how it was prepared and how it was checked in the experiments.

2. Related Work

House-price prediction has been studied using models and machine learning methods. Linear Regression is often used as a starting point because it shows how different features relate to an outcome like price. Ridge and Lasso are versions of linear models that add a penalty to reduce overfitting, which helps when many features are involved. Decision Trees and Random Forests can capture relationships and interactions between features. K-Nearest Neighbours predicts prices by looking at houses in the data. Support Vector Regression and neural networks are options for handling complex patterns.When we turn values into categories classification methods come in handy. Logistic Regression, Bayes, Decision Trees, k-Nearest Neighbours Support Vector Machines, Perceptrons and neural networks all handle binary classification in different ways. How well they work depends on how the featuresre chosen, how balanced the classes are the settings used and what we use to judge their performance.Reducing the number of features is important when we have many. Principal Component Analysis changes correlated features into uncorrelated ones sorted by how much variation they explain.. Just keeping most of the variance doesn't always mean better predictions.

This study compares regression and classification methods using housing data from Bengaluru. It also tests a price-persquare-foot baseline looks at PCA for reducing features and checks how errors vary across different areas and property types.The findings should be seen in the context of existing research, on learning, ensemble methods, model validation, classification evaluation and dimensionality

reduction. Academic references are listed in Section 8. Please check these against the papers before submitting.

3. Dataset and Preprocessing

3.1 Source and Description

The dataset used in this study is the Bengaluru house-price dataset published by Amitabhajoy on Kaggle.
Dataset URL:
https://www.kaggle.com/datasets/amitabhajoy/bengaluru house-price-data
The dataset contains property listings and attributes including area type, location, total floor area, property configuration, bathroom count, balcony count, availability and price. The price variable is expressed in lakh rupees. Is used as the regression target.The original dataset contains [N₀] records and [P₀] variables. After data cleaning and feature construction [N] records remain for modelling. These values must be filled in from the executed notebook.

	Variable	Type	Missing %	Unique
2	area_type	object	0	4
3	availability	object	0	81
4	location	object	0	1305
5	size	object	0.1	31
6	society	object	41.3	2688
7	total_sqft	object	0	2117
8	bath	float64	0.5	19
9	balcony	float64	4.6	4
10	price	float64	0	1994

Table 1. Variables in the Bengaluru housing dataset missing values and their use

3.2 Missing Values

Missing values are examined before model development to understand the completeness of the dataset. The percentage of missing observations is calculated separately for each variable.

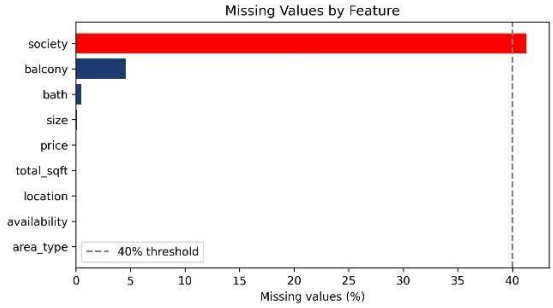


Figure 1. Percentage of missing values per variable in the Bengaluru housing dataset

The figure identifies variables with missingness. The society variable is removed because 41% of its values are missing in the supplied dataset. The 40% threshold is used as a reference rather than as a universal rule for feature removal.

Records with missing or unparseable fields—floor area, property configuration, price or location—are removed. The number of records removed is [N_missing]. Missingness in fields is handled according to the preprocessing pipeline.

The missing-value analysis is important because removing records may change the distribution of properties represented in the sample.

3.3 Feature Construction, Outliers and Scaling

The original floor-area field contains values, ranges and values expressed in different measurement units. These are converted into feet. For ranges the midpoint is used as an approximate floor area. Recognised units, including metres, square yards, acres, cents, guntha, perch and grounds are converted using their corresponding conversion factors.

The BHK configuration is extracted from the original size field. Location strings are stripped of leading and trailing whitespace. A binary feature, ready is created to indicate whether a property is listed as ready to move into.

Two heuristic screening rules are applied: floor area per BHK must be least 300 square feet and the bathroom count must not exceed the BHK count by more than two. These rules are intended to identify implausible records but they may also exclude legitimate properties. The number of records removed by the rules is [N_rules] and the final modelling sample contains [N_final] records.Outliers are examined using the range (IQR). For each variable observations outside the interval defined by the first quartile minus 1.5 times the IQR and the third quartile plus 1.5 times the IQR are counted. The resulting percentages are reported before and after the screening rules.During model preparation numerical features are imputed using training-set medians, clipped to the 99th percentiles estimated from training data and standardised. Categorical features are onehot encoded. All learned preprocessing parameters must be fitted on training data only. Then applied to validation and test data.

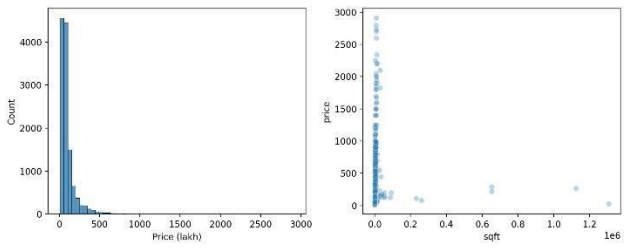


Figure 2. Mean property-price exploratory analysis

For this study Figure 2 consists of two panels: (a) the distribution of listed property prices and (b) the relationship between total floor area and listed price.

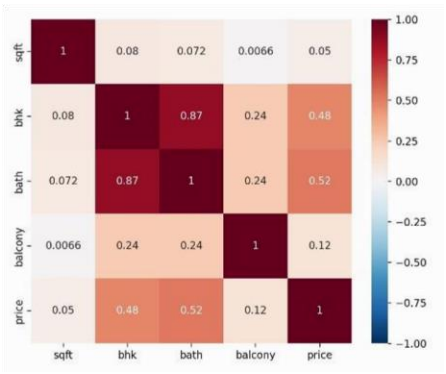


Figure 3. Correlation matrix of numerical housing variables

The correlation matrix includes floor area, BHK, bathrooms, balconies and listed price. Pearson correlation measures the strength and direction of linear association between variables. Correlation does not establish causality and a weak linear correlation does not rule out a relationship.

3.4 Feature Selection and PCA

The selected predictors are floor area, BHK, bathrooms, balconies, readiness status, location and area type. The listed price is excluded from the predictor matrix for regression. The affordability label is excluded from the classification predictors.

Price-per-foot is not included as a predictor because it is derived from the target price and floor area. Using it to predict the same price would introduce target leakage.

Lasso Regression is used to examine regularised linear coefficients while Random Forest feature importance is used to assess the contribution of transformed features to the fitted forests predictions. These analyses provide model-interpretation rather than proof of causal importance. Principal Component Analysis is investigated to determine how dimensionality reduction affects regression performance. The cumulative explained variance is used to determine the number of components needed to retain 90% and 95% of the training feature variance.

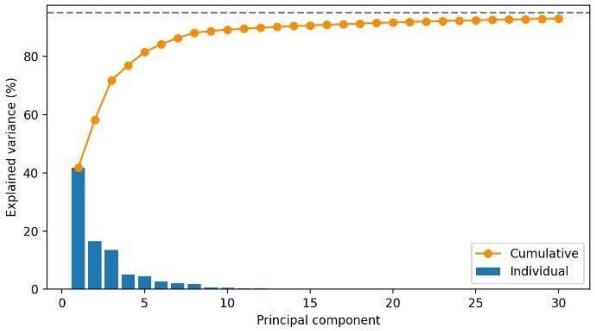


Figure 4. Individual and cumulative explained variance of components

The PCA comparison uses the train–test split for the original and reduced feature representations. Preprocessing and PCA must be fitted exclusively on training data. Any conclusions about improvements, in performance should be based on the test metrics not on explained variance alone.

4. Methodology

4.1 Regression Models

The regression task predicts listed property prices in lakh rupees using selected property features. Eight models are compared:

- Linear Regression (Ordinary Least Squares)
- Ridge Regression
- Lasso Regression
- Decision Tree Regression
- k-Nearest Neighbours Regression
- Support Vector Regression with a radial basis function kernel
- Random Forest Regression
- Multilayer Perceptron Regression

Linear Regression serves as a linear baseline. Ridge and Lasso add regularization to prevent overfitting. Decision Trees and Random Forests capture patterns in the data. KNN estimates prices by looking at properties. Support Vector Regression uses an RBF kernel to model relationships. The MLP model is a network with two hidden layers—64 units in the first and 32 in the second.

To handle the distribution of prices a logarithmic transformation is applied to the target during training. After predicting the results are converted back to the price scale for evaluation.

The regression models are compared to a baseline: multiplying floor area by the median rupees-per-square-foot from the training data. This gives a benchmark to see how much better the machine learning models perform.

4.2 Classification Models

The classification task sorts each property into one of two groups based on its listed price. A property is labeled affordable if it costs ₹45 lakh or less. If it costs more it's labeled not affordable.

The classification models tested are: Logistic Regression, Gaussian Naive Bayes, Decision Tree Classification, kNearest Neighbours, Perceptron, Support Vector

Classification with an RBF kernel and Multilayer Perceptron Classification.
This ₹45 lakh threshold is a choice for this study. It's not a measure of housing affordability because it doesn't consider things like household income, loan interest or maintenance costs.Hyperparameters are chosen using cross-validation on the training data. The final model is tested on a test set. When choosing the model we should rely on cross-validation results, not test-set F1-scores to avoid overfitting to the test data.

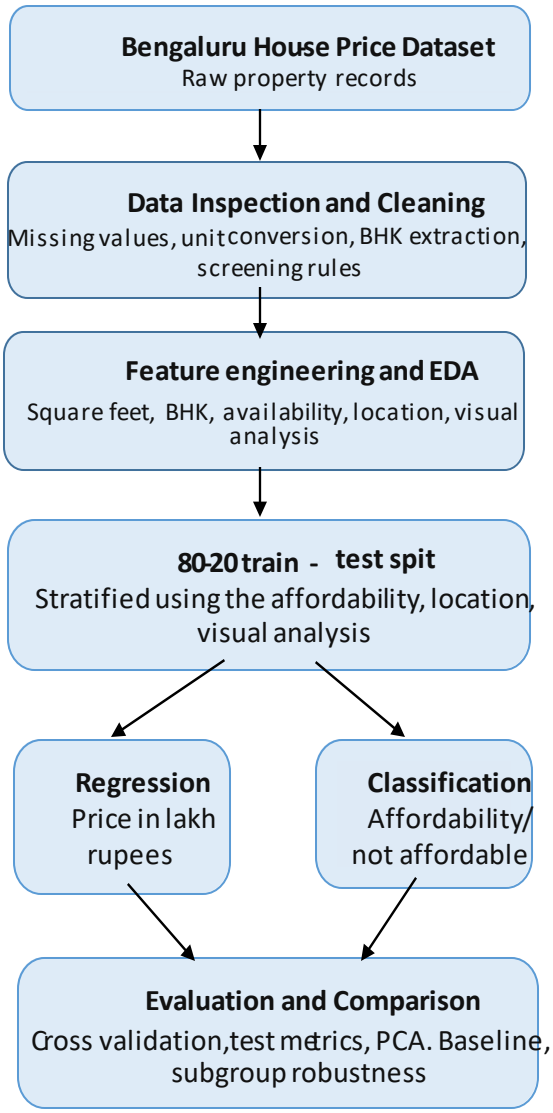


Figure 5. Overview of the proposed data preprocessing, feature engineering, modelling, and evaluation framework.

4.3 Evaluation Metrics

For regression we use Mean Absolute Error (MAE) Root Mean Squared Error (RMSE) and R^2.

MAE shows the size of prediction errors in lakh rupees. RMSE gives weight to larger errors. R^2 compares how well the model fits the data versus predicting the average. Lower MAE and RMSE mean accuracy. Higher R^2 means performance.
For classification we measure accuracy (classified cases) precision (how many predicted affordable homes are truly affordable) recall (how many real affordable homes were found) and F1-score (the balance between precision and recall). ROC-AUC shows how well the model can tell apart the two classes across thresholds.
The same test set is used to evaluate both regression and classification models. This test set is not used during hyperparameter tuning.

5. Experimental Setup

The experiments are implemented in Python using Google Colab. The main libraries used are pandas, NumPy, Matplotlib, Seaborn and scikit-learn. The random seed is set to 42 to ensure consistency across runs.
The cleaned dataset is split into training and test subsets with an 80:20 ratio. Stratification is applied based on the affordability label to preserve the class distribution closely as possible. The final counts are [N_train] training records and [N_test] test records.
Numerical features are handled by imputing missing values then winsorizing based on percentile values from the training set and finally standardizing the data. Categorical features are one-hot encoded with rare categories grouped when the installed scikit-learn version supports this functionality.

For regression models performance is evaluated using five-fold shuffled cross-validation on the training set. Hyperparameter tuning is done using -validated negative RMSE as the scoring metric. For classification models five-cross-validation is used with F1-score as the criterion for selecting the best hyperparameters.

The provided implementation uses shuffled Kfold validation, not forward-chaining validation. Therefore the paper must not state that forward-chaining cross-validation was used. Since the dataset is not structured as a daily time series in this analysis a random holdout evaluation is suitable for the experiment. However a time-based split could be considered if reliable listing dates were available.

Model	Grid searched	Selected
Linear (OLS)	-	-
Ridge	{'m__regressor__alpha': [1, 10, 100]}	{'alpha': 1}
Lasso	{'m__regressor__alpha': [0.001, 0.01, 0.1]}	{'alpha': 0.001}
Decision tree	{'m__regressor__max_depth': [6, 10, 14], 'm__regressor__min_samples_leaf': [5, 20]}	{'max_depth': 14, 'min_samples_leaf': 20}
k-NN	{'m__regressor__n_neighbors': [5, 15, 31], 'm__regressor__weights': ['uniform', 'distance']}	{'n_neighbors': 5, 'weights': 'uniform'}
SVR (RBF)	{'m__regressor__C': [10, 100], 'm__regressor__epsilon': [0.05, 0.2]}	{'C': 10, 'epsilon': 0.2}
Random forest	{'m__regressor__n_estimators': [200], 'm__regressor__min_samples_leaf': [1, 3]}	{'min_samples_leaf': 3, 'n_estimators': 200}
MLP (64-32)	{'m__regressor__alpha': [0.0001, 0.01, 1]}	{'alpha': 0.0001}
Logistic regressior	{'m__C': [0.1, 1, 10], 'm__class_weight': [None, 'balanced']}	{'C': 10, 'class_weight': 'balanced'}
Gaussian NB	-	-
Decision tree	{'m__max_depth': [4, 8, 12], 'm__class_weight': [None, 'balanced']}	{'class_weight': 'balanced', 'max_depth': 12}
k-NN	{'m__n_neighbors': [5, 15, 31]}	{'n_neighbors': 5}
Perceptron	-	-
SVM (RBF)	{'m__C': [0.1, 1, 10]}	{'C': 10}
MLP (64-32)	{'m__alpha': [0.0001, 1]}	{'alpha': 0.0001}

Table 2. Hyperparameter. Values selected by five-fold cross-validation

The table shows the hyperparameter values that were tested and the final values selected for each model.
The baseline model is computed using the training data. For regression the baseline price estimate is the floor area of the

property multiplied by the rupees-per-square-foot ratio, from the training set. For classification the baseline affordable label is determined using the ₹45 lakh threshold based on the regression baseline price estimate.

The software environment must be recorded after execution, including the versions of Python, scikit-learn, pandas and NumPy. These details help ensure reproducibility of the results.

6. Results

6.1 House-Price Regression

The regression experiment compares the eight machinelearning models with the median price-per-square-foot baseline. The comparison uses MAE, RMSE, R^2, and crossvalidated RMSE.

Model	MAE	RMSE	R2	CV-RMSE
Linear (OLS)	35.729	95.35	0.522	95.309
Ridge	35.671	95.181	0.523	95.289
Lasso	39.039	101.235	0.461	97.118
Decision tree	35.192	93.013	0.545	98.978
k-NN	33.23	88.509	0.588	97.616
SVR (RBF)	31.247	90.089	0.573	94.373
Random forest	32.112	88.54	0.588	93.357
MLP (64-32)	30.238	83.161	0.636	92.231
Baseline (median Rs/sqft)	83.119	1375.219	-98.486	

Table 3. House-price regression on the held-out test set

The best model under the RMSE criterion is **MLP (64-32)**, with a test RMSE of **92.231 lakh rupees**, an MAE of **30.238 lakh rupees**, and an R^2 of **0.636**.

These results should be compared with the baseline RMSE of **1375.219 lakh rupees**. If the selected model has lower RMSE than the baseline, the result indicates that its predictions improve on the chosen baseline for this test set. If the baseline performs better, that result should be reported transparently.

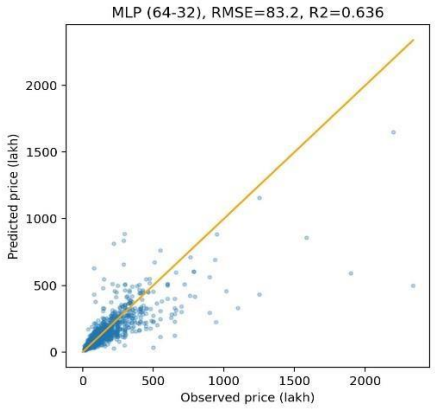


Figure 6. Observed versus predicted house prices for the selected regression model

The horizontal axis represents observed prices and the vertical axis represents predicted prices. Points near the diagonal indicate more accurate predictions. Deviations from the diagonal illustrate prediction errors, particularly for properties with unusually high or low prices.

The final discussion should identify the model with the lowest observed test RMSE and explain the pattern visible in the prediction plot. Claims about model superiority must be supported by the actual results.

6.2 Affordability Classification

The classification experiment compares seven algorithms for identifying properties priced at or below ₹45 lakh. Performance is measured using accuracy, precision, recall, F1-score, and ROC-AUC.

Model	Acc	Prec	Recall	F1	AUC
Logistic regression	0.834	0.556	0.873	0.68	0.924
Gaussian NB	0.381	0.241	0.96	0.385	0.6
Decision tree	0.805	0.51	0.885	0.647	0.891
k-NN	0.874	0.701	0.655	0.677	0.891
Perceptron	0.818	0.62	0.252	0.358	0.798
SVM (RBF)	0.886	0.763	0.631	0.691	0.924
MLP (64-32)	0.892	0.789	0.637	0.705	0.933
Baseline	0.841	0.792	0.288	0.422	

Table 4. Affordability classification on the held-out test set

The selected classifier should be identified according to its cross-validation F1-score, and its held-out test metrics should then be reported.

The best classifier is **MLP (64-32)**, with a test accuracy of **89%** , precision of **0.789**, recall of **0.637**, F1-score of **0.705**, and ROC-AUC of **0.933**.

The precision and recall values should be considered together. A model with high precision identifies affordable properties more selectively, whereas a model with high recall identifies a larger proportion of properties below the price threshold. The appropriate balance depends on the intended use of the screening system.

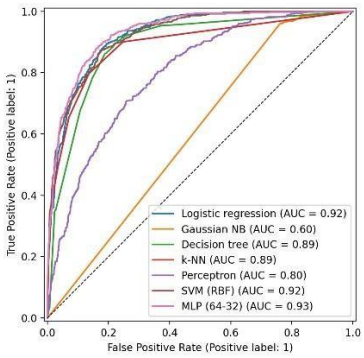


Figure 7. ROC curves of the seven affordability classifiers

The ROC plot compares true-positive rates with falsepositive rates across classification thresholds. Curves closer to the top-left generally indicate stronger discrimination. The

ROC-AUC values should be reported alongside the curves to support quantitative comparison.

A confusion matrix may be included as an additional diagnostic if the page limit permits. It should be generated from the selected classifier after model selection is completed using training cross-validation.

6.3 Effect of PCA and Robustness Across Locations

PCA is evaluated by comparing Linear Regression and k-NN with and without dimensionality reduction. Table 5 reports MAE, RMSE, and R^2 for the four configurations.

Setting	MAE	RMSE	R2
OLS, all features	35.729	95.35	0.522
OLS, PCA 95%	39.222	101.207	0.461
k-NN (k=15), all features	33.765	92.014	0.555
k-NN (k=15), PCA 95%	34.553	92.529	0.55

Table 5. Regression performance with and without PCA

The number of components required to retain 90% and 95% of the training feature variance is **13%** and **56%**, respectively. The results show that PCA **reduced** predictive performance for **linear regression** and **did not materially change** predictive performance for **k-NN**, based on the observed error metrics.

PCA should not be considered automatically beneficial. Although dimensionality reduction may simplify the feature representation, it can also discard information useful for price prediction.

Robustness is further assessed across locations according to their representation in the training set. Locations are grouped as unseen, rare (one to nine training records), and frequent (at least ten training records). Test errors are also examined across BHK configurations, with configurations above five grouped together.

7. Discussion

The study provides a comparative evaluation of regression and classification models for Bengaluru residential property listings. The regression task estimates listed prices, while the classification task identifies properties below a predefined threshold. These tasks provide complementary perspectives on the same data but address different prediction objectives.

The relative performance of the regression models should be interpreted using both MAE and RMSE. A smaller MAE indicates lower average absolute errors, whereas a comparatively high RMSE may indicate substantial errors for a subset of properties. R^2 adds information about the model's fit relative to a constant-mean reference.

Classification performance should be interpreted in terms of the intended screening objective. Accuracy alone can be insufficient if the classes are imbalanced. Precision, recall, F1-score, and ROC-AUC provide a broader assessment of the classifier's behaviour.

The PCA comparison examines whether a reduced feature representation retains enough information for accurate prediction. The results should establish whether dimensionality reduction is useful for the selected models rather than assuming that fewer features necessarily lead to better performance.

The location-based analysis is particularly relevant because property prices can vary substantially across neighbourhoods. Differences in training-data representation and within-location price variation may contribute to unequal prediction errors. Nevertheless, the available attributes do not capture every factor affecting residential value.

The affordability classification also has important limitations. A property below ₹45 lakh is not necessarily affordable for every household, and a property above that price is not necessarily unaffordable to every buyer. Household income, loan eligibility, interest rates, maintenance, and commuting costs are outside the scope of this threshold-based experiment.

Finally, the findings apply to the cleaned dataset and the specified evaluation procedure. Further validation would be required before applying the models to current listings, verified sale transactions, or other housing markets.

8. Conclusion and Future work

This paper introduces the machine learning approach used in Bengaluru for predicting house prices and classifying houses into different classes of affordability based on threshold pricing.

The experiments performed during this research encompass data pre-processing, feature creation, exploratory data analysis, regression and classification model building, comparison of PCAs, and robustness test based on locations and BHK types.

The results of the final experiment show that MLP (64-32) is the algorithm with the minimum test RMSE of 83.16 lakh rupees. The comparison of the more complicated models to the price per square foot model will show if these models can perform better than the baseline one on the test set. The PCA test checks if the dimensionality reduction helps in improving the regression models chosen for the prediction task. The robustness test demonstrates the variations in the predictions across the available location groups.

Future research could consider time-based testing, more complicated approaches to handling missing values, inclusion of other features related to neighborhoods, model explanations and affordability measures based on income levels of households.

9. Declaration

Data availability: The Bengaluru house-price dataset is available at

https://www.kaggle.com/datasets/amitabhajoy/bengaluru house-price-data.
The analysis code, generated figures, and tables may be shared through a public repository after validation and documentation.

Conflict of interest: The author declares no known competing financial interests or personal relationships that could have influenced this study.

Funding: This study received no external funding .

Use of AI tools: This project, Bengaluru House Price Prediction was created with help from AI tools that assisted with coding, debugging and improving the machine learning workflow. The experiments and results shown in this project were produced by running the Jupyter Notebook, named named Bengaluru_House_Price_Prediction.ipynb. Every AI suggestion was. The code, results and explanations were checked as thoroughly as possible. I fully own the work that is submitted. I am responsible, for the accuracy of the reported results and the final content.

10. References

[1] Amitabhajoy, "Bengaluru House Price Data," Kaggle. Available: https://www.kaggle.com/datasets/amitabhajoy/bengaluru house-price-data. Accessed: [date].

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009. [4] I. T. Jolliffe and J. Cadima, "Principal component analysis: A review and recent developments," *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 2016.

[5] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233–240.

Appendix A. Per-Location Test Error

Table A1. House-price RMSE by location on the heldout test set

Insert tableA1_per_location.csv on Page 6.

The table reports the number of training records, number of test records, mean listed price, selected-model RMSE, baseline RMSE, and whether the selected model's RMSE exceeds the baseline RMSE. Results are shown for the twenty locations with the largest test-set representation.

The appendix provides a more detailed view of model reliability across neighbourhoods. Locations with small test

loc	n_train	n_test	mean_price	RMSE_model	RMSE_baseline	worse_than_baseline
Whitefield	434	100	112.9	92.6	90.2	TRUE
Sarjapur Road	307	81	118.4	66.1	72.5	FALSE
Electronic City	229	66	51.2	12.3	17.4	FALSE
Kanakpura Road	209	55	72.3	15.5	24.2	FALSE
Thanisandra	184	50	75.2	18.8	23.1	FALSE
Uttarahalli	145	35	54.3	10.2	22	FALSE
Yelahanka	171	34	82.3	42.5	41	TRUE
Bannerghatta Road	117	34	92.8	35.6	52.6	FALSE
Marathahalli	138	32	122.3	30.9	43.7	FALSE
Hebbal	144	31	182	127.6	122.3	TRUE
Haralur Road	111	29	82.3	32.8	41.5	FALSE
Rajaji Nagar	73	26	236.8	51.4	170.5	FALSE
Raja Rajeshwari Nagar	147	23	59.6	22.4	21.3	TRUE
7th Phase JP Nagar	124	22	105.1	34.2	53.3	FALSE
Hennur Road	127	22	93.2	16.9	23.7	FALSE
Kasavanhalli	57	22	90	22.5	29.2	FALSE
Electronic City Phase II	107	19	55.6	56.9	57.2	FALSE
Banashankari	51	18	113.2	33.4	79.8	FALSE
Electronics City Phase 1	68	18	55.8	7.6	7.6	FALSE
8th Phase JP Nagar	36	17	61	24.4	17.4	TRUE

counts should be interpreted cautiously because a few prediction errors can substantially affect their RMSE values.